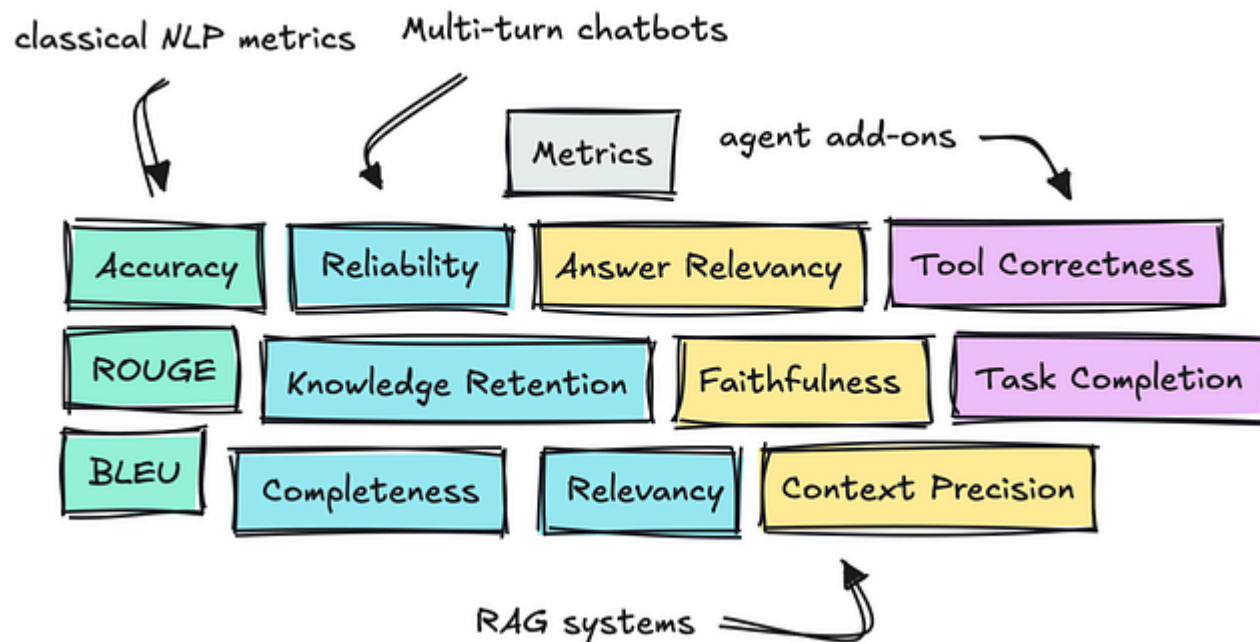


[< Go to the original](#)

# Agentic AI: Working with Evaluations

What metrics to track, and frameworks that help



**Ida Silfverskiöld**

Follow



Data Science Collective a11y-light ~14 min read · August 4, 2025 (Updated: August 4, 2025) ·

## Building LLM Applications

*If you're not a member and you would like to read this story click [here](#).*

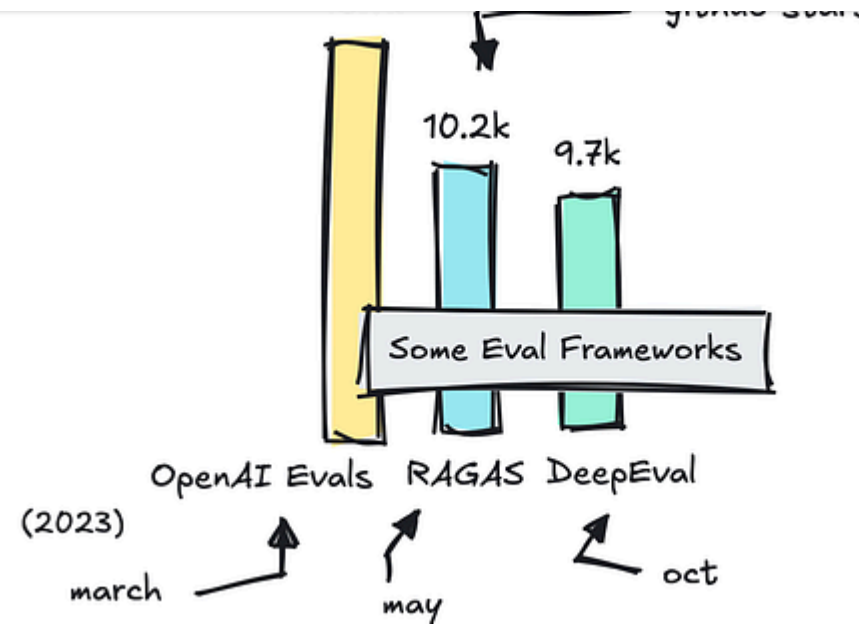
Agentic Evals is about testing your LLM application to make sure it performs consistently.

It's not the most exciting topic, but more and more companies are paying attention. So it's worth digging into to understand which metrics you need to track to actually measure that performance.

It also helps to have proper evals in place anytime you push changes, to make sure things don't go haywire.

This article will be surface level, but it will give you some intel on why we still mention metrics like BLEU and ROUGE, even though we now also talk about something called LLM-as-a-judge.

It will also give you insight into the kinds of metrics we track for multi-turn chatbots, RAG, and agentic applications, along with how we test for those metrics.



The different popular frameworks we'll cover later around performing evals | Image by author

It wraps up with a comparison of different eval frameworks that can help you out, such as DeepEval, RAGAS, and OpenAI's evals library.

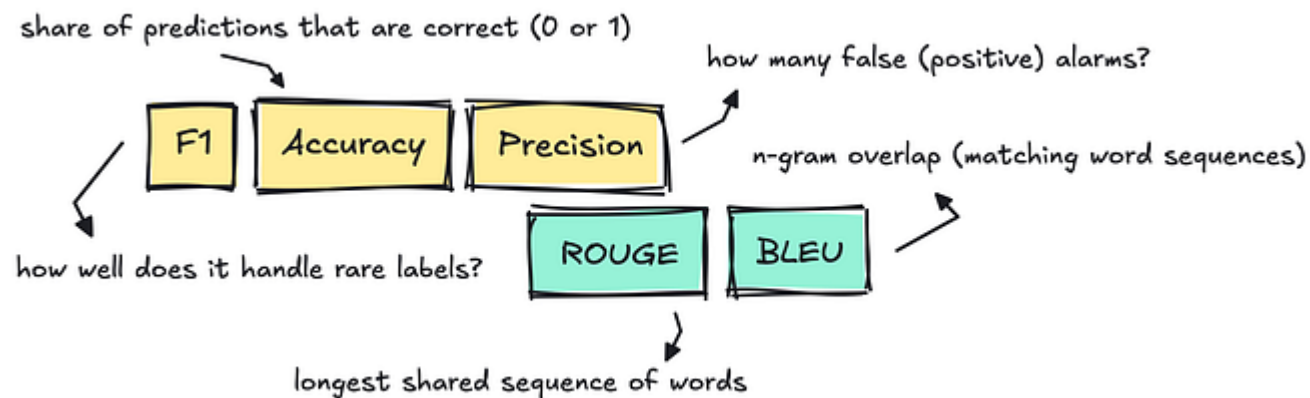
The first part gives you a bit of history, which you can skip if you're already familiar. The second part digs into how to work with evals for more advanced LLM applications.

## What we did before

If you're not, it's good to know what the earlier metrics like accuracy and BLEU were originally used for and how they work, along with understanding how we test for public benchmarks like MMLU.

## Evaluating NLP tasks

When we evaluate traditional NLP tasks such as classification, translation, summarization, and so on, we turn to traditional metrics like accuracy, precision, F1, BLEU, and ROUGE.



Take classification, for example, where the task is to assign each text a single label. To test this, we can use accuracy by comparing the label assigned by the model to the reference label in the eval dataset to see if it got it right.

It's very clear-cut: if it assigns the wrong label, it gets a 0; if it assigns the correct label, it gets a 1.

This means if we build a classifier for a spam dataset with 1,000 emails, and the model labels 910 of them correctly, the accuracy would be 0.91.

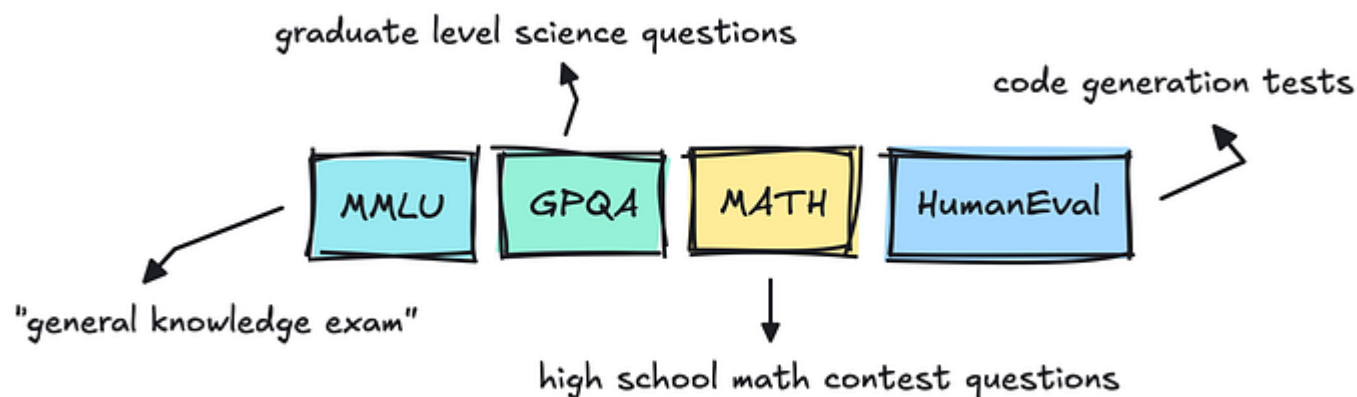
For text classification, we often also use F1, precision, and recall.

When it comes to NLP tasks like summarization and machine translation, people often used ROUGE and BLEU to see how closely the model's translation or summary lines up with a reference text.

Both scores just count overlapping n-grams: the more shared word chunks, the higher the score. This is pretty simplistic, since if the outputs use different wording, it will score low.

## LLM benchmarks

If you've watched the news, you've probably seen that every time a new version of a large language model gets released, it follows a few benchmarks: MMLU Pro, GPQA, or Big-Bench.



These are generic evals for which the proper term is really "benchmark" and not evals (which we'll cover later).

Although there's a variety of other evaluations done for each model, including for toxicity, hallucination, and bias, the ones that get most of the attention are more

Datasets like MMLU are multiple-choice and have been around for quite some time. I've actually skimmed through it before and seen how messy it is.

Some questions and answers are quite ambiguous, which makes me think that LLM providers will try to train their models on these datasets just to make sure they get them right.

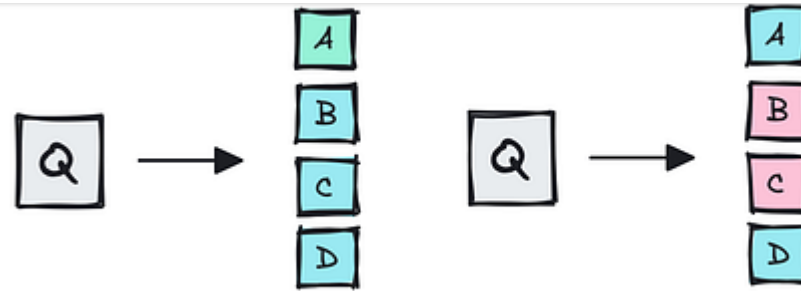
This creates some fear in the general public that most LLMs are just overfitting when they do well on these benchmarks and why there's a need for newer datasets and independent evaluations.

## LLM scorers

To run evaluations on these datasets, you can usually use accuracy and unit tests. However, what's different now is the addition of something called LLM-as-a-judge.

To benchmark the models, teams will mostly use traditional methods.

So as long as it's multiple choice or there's just one right answer, there's no need for anything else but to compare the answer to the reference for an exact match.

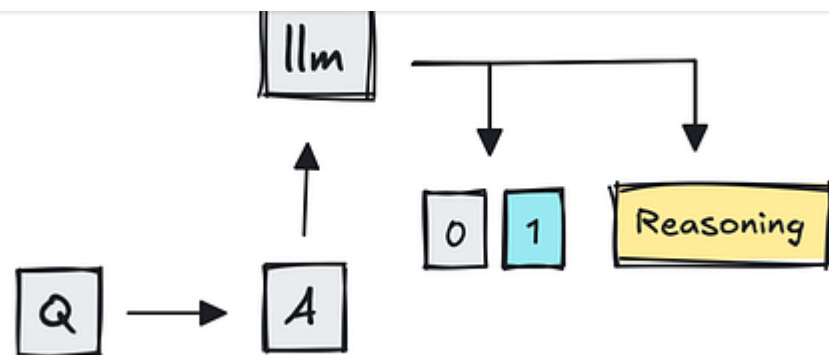


Multiple answer questions we simply parse the responses | Image by author

This is the case for datasets such as MMLU and GPQA, which have multiple choice answers.

For the coding tests (HumanEval, SWE-Bench), the grader can simply run the model's patch or function. If every test passes, the problem counts as solved, and vice versa.

However, as you can imagine, if the questions are ambiguous or open-ended, the answers may fluctuate. This gap led to the rise of "LLM-as-a-judge," where a large language model like GPT-4 scores the answers.



We let an LLM score an answer with reasoning | Image by author

MT-Bench is one of the benchmarks that uses LLMs as scorers, as it feeds GPT-4 two competing multi-turn answers and asks which one is better.

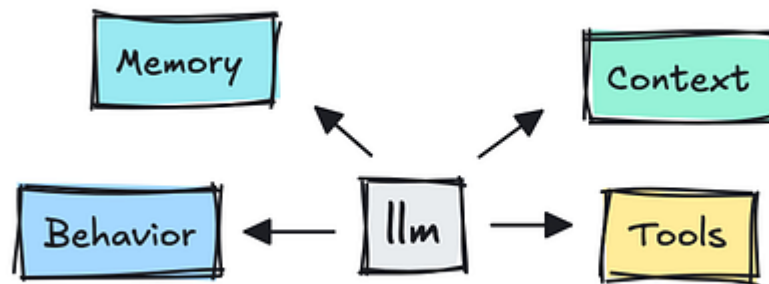
Chatbot Arena, which use human raters, I think now scales up by also incorporating the use of an LLM-as-a-judge.

*For transparency, you can also use semantic rulers such as BERTScore to compare for semantic similarity. I'm glossing over what's out there to keep it condensed.*

So, teams may still use overlap metrics like BLEU or ROUGE for quick sanity checks, or rely on exact-match parsing when possible, but what's new is to have another large language model judge the output.

## What we do with LLM apps

The primary thing that changes now is that we're not just testing the LLM itself but the entire system.



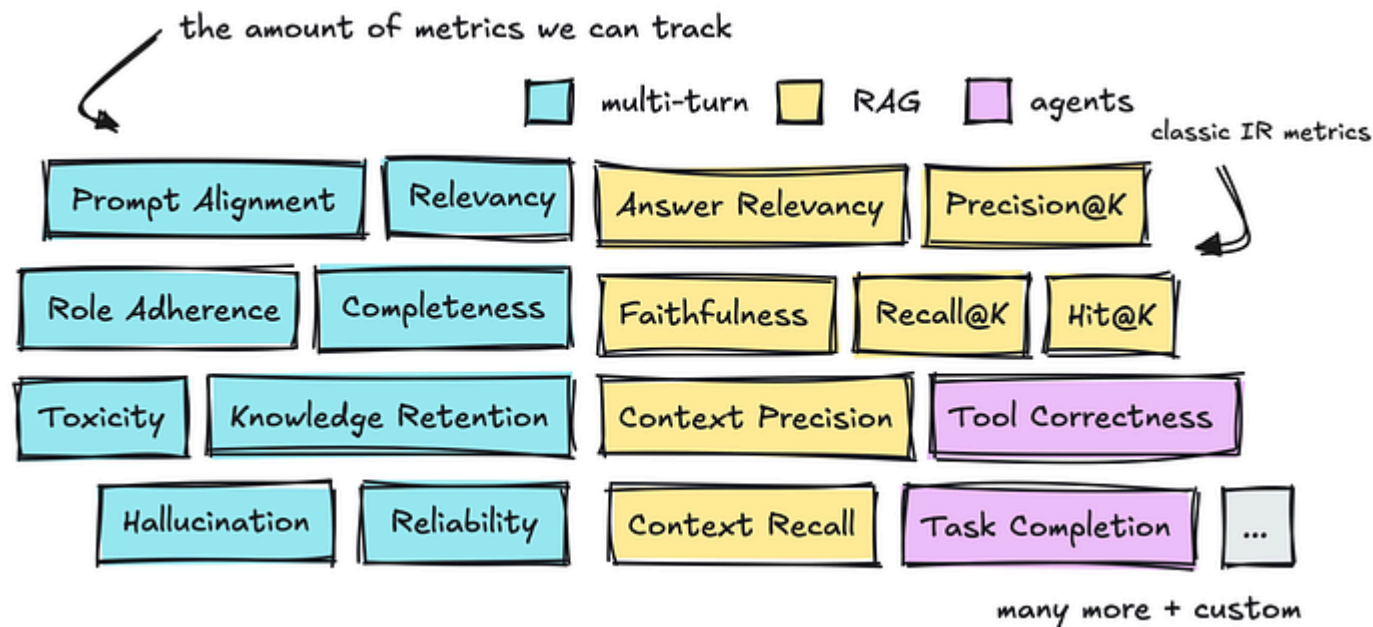
We're evaluating the outputs of an entire system instead of just the LLM | Image by author

When we can, we still use programmatic methods to evaluate, just like before.

For more nuanced outputs, we can start with something cheap and deterministic like BLEU or ROUGE to look at n-gram overlap, but most modern frameworks out there will now use LLM scorers to evaluate.

metrics we can turn to.

You can see below the massive amount of metrics that have already been defined in these three areas.



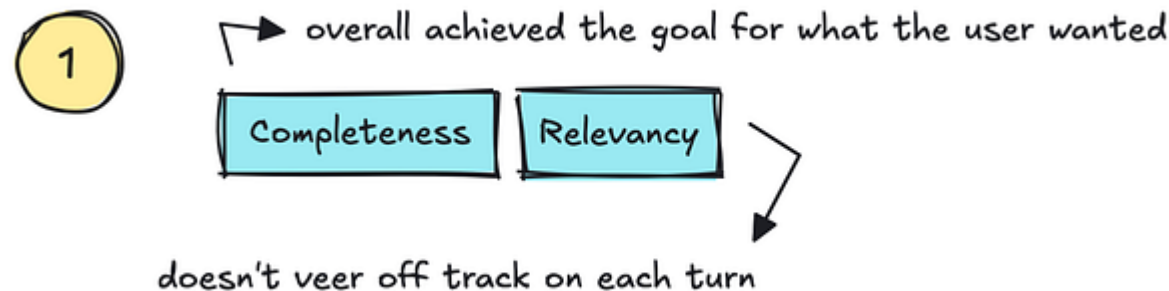
We'll talk about all of these briefly before moving on to the different frameworks that help us out.

## Multi-turn conversations

When we interact with chatbots, we want the conversation to feel natural, professional, and for it to remember the right bits. We want it to stay on topic throughout the conversation and actually answer the thing we asked.

There are quite a few standard metrics people track here. The first we can talk about are **Relevancy/Coherence** and **Completeness**.

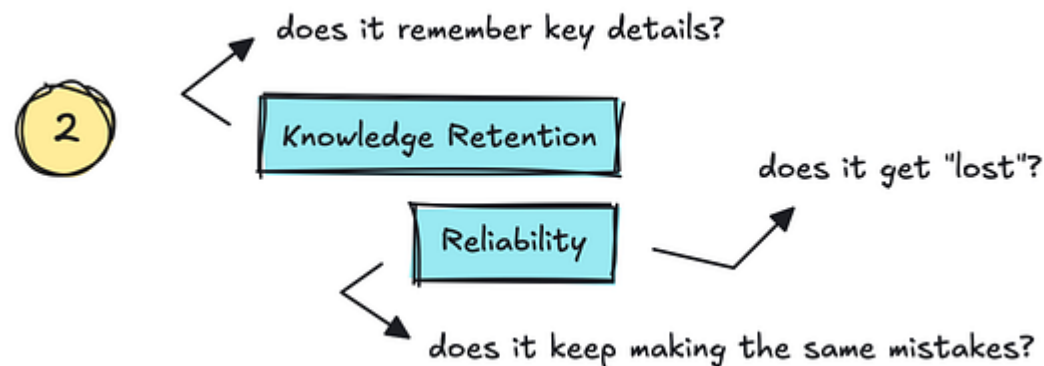
**Relevancy** is a metric that should track if the LLM appropriately addresses the user's query and stays on topic, whereas Completeness is high if the final outcome actually addresses the user's goal.



providing high "self-service rates."

The second part, as I call it, is **Knowledge Retention** and **Reliability**.

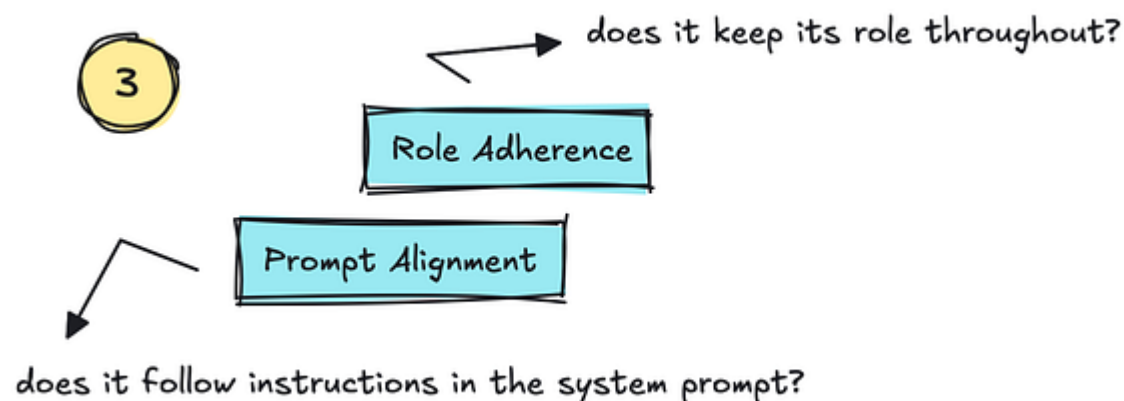
That is: does it remember key details from the conversation, and can we trust it not to get "lost"? It's not just enough that it remembers details. It also needs to be able to correct itself.



This is something we see in vibe coding tools. They forget the mistakes they've made and then keep making them. We should be tracking this as low **Reliability** or **Stability**.

context window. So the things we track here are essentially about how to work on good context engineering.

The third part we can track is **Role Adherence** and **Prompt Alignment**. This tracks whether the LLM sticks to the role it's been given and whether it follows the instructions in the system prompt.



Next are metrics around safety, such as **Hallucination** and **Bias/Toxicity**.

**Hallucination** is important to track but also quite difficult. People may try to set up web search to evaluate the output, or they split the output into different claims that are evaluated by a larger model (LLM-as-a-judge style).

original answer and how many times it diverges.

For **Bias/Toxicity**, you can use other NLP methods, such as a fine-tuned classifier.

Other metrics you may want to track could be custom to your application, for example, code correctness, security vulnerabilities, JSON correctness, and so on.

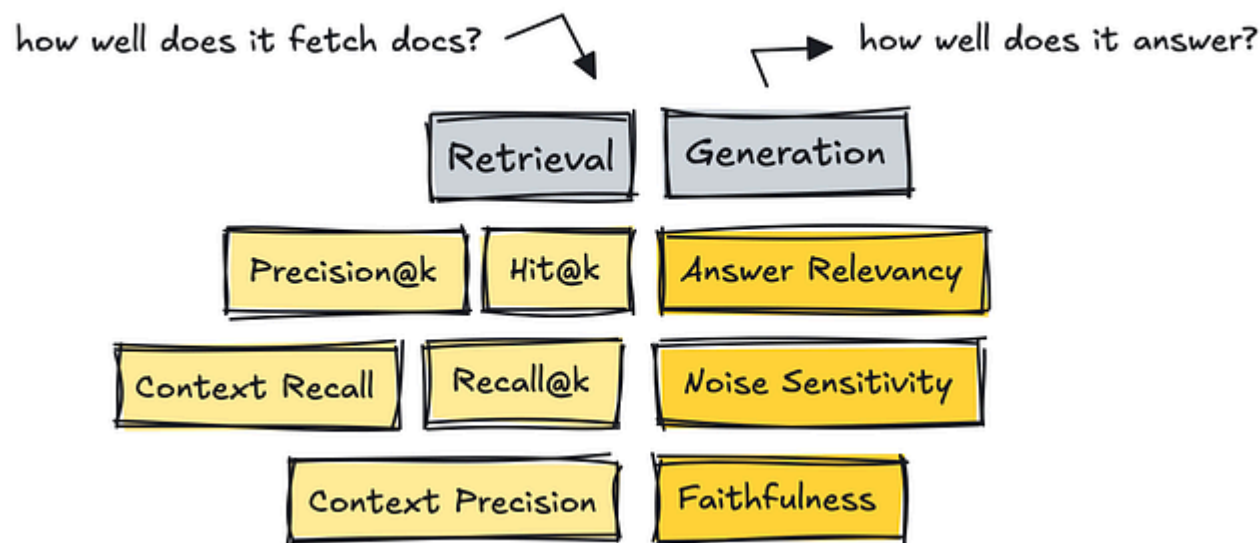
As for how to do the evaluations, you don't always have to use an LLM, although in most of these cases the standard solutions do.

In cases where we can extract the correct answer, such as parsing JSON, we naturally don't need to use an LLM. As I said earlier, many LLM providers also benchmark with unit tests for code-related metrics.

It goes without saying that LLMs used to judge aren't always super reliable, just like the applications they measure, but I don't have any numbers for you here, so you'll have to hunt for that on your own.

## Retrieval Augmented Generation (RAG)

With RAG systems, we need to split the process into two: measuring retrieval and generation metrics separately.



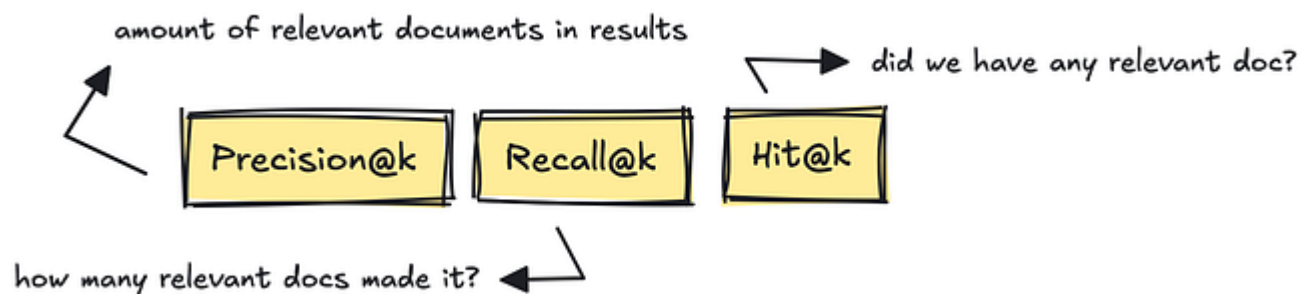
The first part to measure is retrieval and whether the documents that are fetched are the correct ones for the query.

If we get low scores on the retrieval side, we can tune the system by setting up better chunking strategies, changing the embedding model, adding techniques

To measure retrieval, we can use older metrics that rely on a curated dataset, or we can use reference-free methods that use an LLM as a judge.

I need to mention the classic IR metrics first because they were the first on the scene. For these, we need "gold" answers, where we set up a query and then rank each document for that particular query.

Although you can use an LLM to build these datasets, we don't use an LLM to measure, since we already have scores in the dataset to compare against.



The most well-known IR metrics are Precision@k, Recall@k, and Hit@k.

least one relevant document made it into the results.

The newer frameworks such as RAGAS and DeepEval avoid these metrics and introduce reference-free, LLM-judge style metrics like Context Recall and Context Precision.

These count how many of the truly relevant chunks made it into the top K list based on the query, using an LLM to judge.

That is, based on the query, did the system actually return any relevant documents, or are there too many irrelevant ones to answer the question properly?

To build datasets for evaluating retrieval, you can mine questions from real logs and then use a human to curate them.

You can also use dataset generators with the help of an LLM, which exist in most frameworks or as standalone tools like YourBench.

If you were to set up your own dataset generator using an LLM, you could do something like below.

```
# Prompt to generate questions
qa_generate_prompt_tmpl = """\
Context information is below.

-----
{context_str}
-----

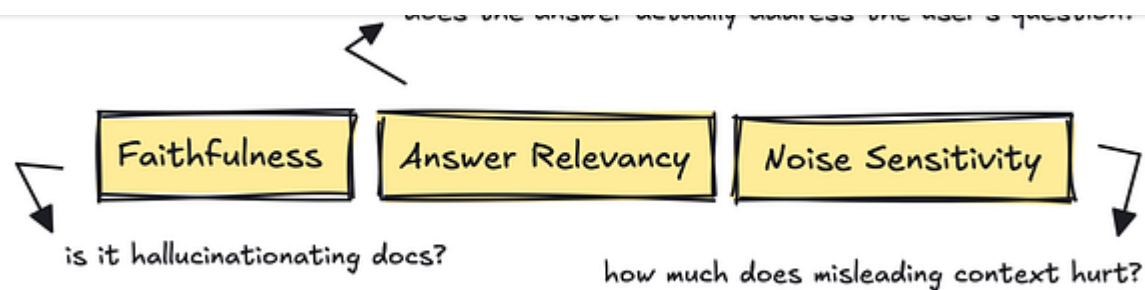
Given the context information and no prior knowledge
generate only {num} questions and {num} answers based on the above context.

...
"""
```

If we turn to the generation part of the RAG system, we are now measuring how well it answers the question using the provided docs.

If this part isn't performing well, we can adjust the prompt, tweak the model settings (temperature, etc.), replace the model entirely, or fine-tune it for domain expertise. We can also force it to "reason" using CoT-style loops, check for self-consistency, and so on.

For this part, RAGAS is useful with its metrics: Answer Relevancy, Faithfulness, and Noise Sensitivity.



These metrics ask whether the answer actually addresses the user's question, whether every claim in the answer is supported by the retrieved docs, and whether a bit of irrelevant context throws the model off course.

If we look at RAGAS, what they likely do for the first metric is ask the LLM to "Rate from 0 to 1 how directly this answer addresses the question," providing it with the question, answer, and retrieved context. This returns a raw 0–1 score that can be used to compute averages.

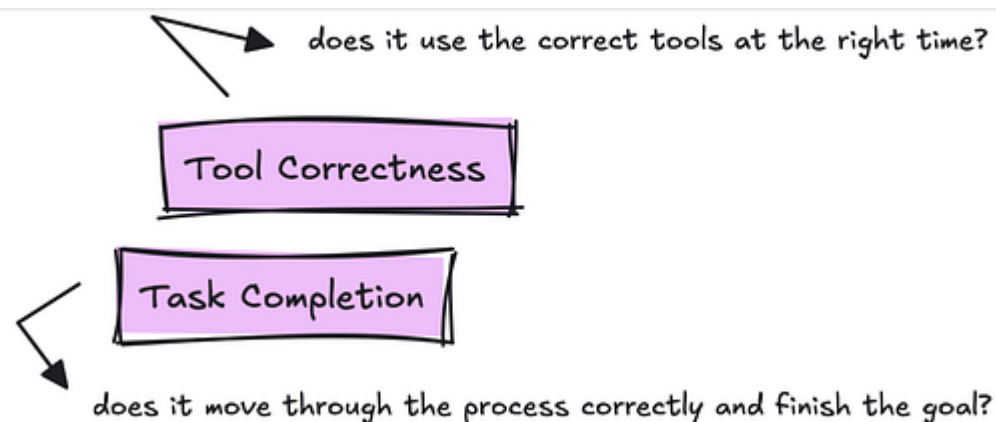
So, to conclude we split the system into two to evaluate, and although you can use methods that rely on the IR metrics you can also use reference free methods that rely on an LLM to score.

## Agents

With agents, we're not just looking at the output, the conversation, and the context.

Now we're also evaluating how it "moves": whether it can complete a task or workflow, how effectively it does so, and whether it calls the right tools at the right time.

Frameworks will call these metrics differently, but essentially the top two you want to track are Task Completion and Tool Correctness.



For tracking tool usage, we want to know if the correct tool was used for the user's query.

We do need some kind of gold script with ground truth built in to test each run, but you can author that once and then use it each time you make changes.

For Task Completion, the evaluation is to read the entire trace and the goal, and return a number between 0 and 1 with a rationale. This should measure how effective the agent is at accomplishing the task.

For agents, you'll still need to test other things we've already covered, depending on your application.

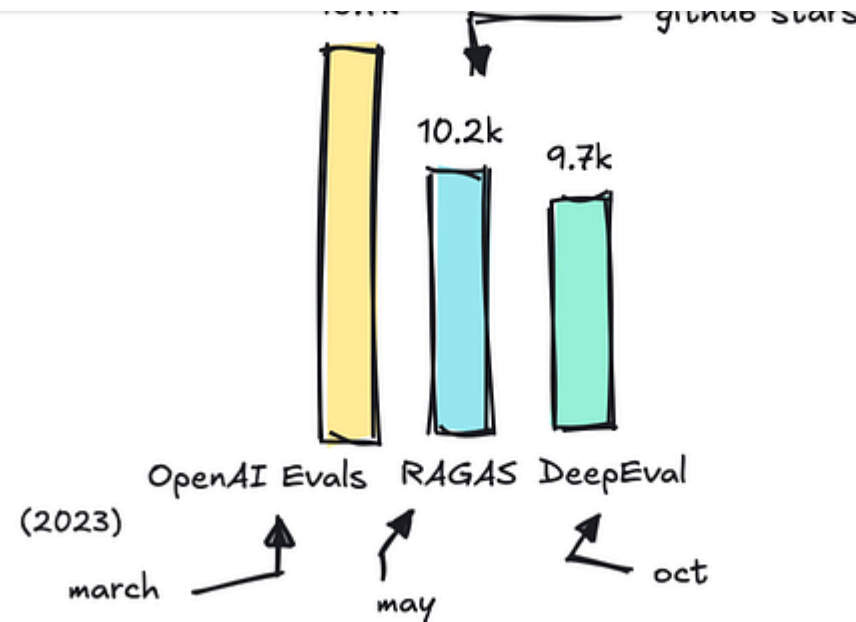
---

*are the best ones to track for your app.*

Next, let's turn to get an overview of the popular frameworks out there that can help you out.

### **Frameworks that help you out**

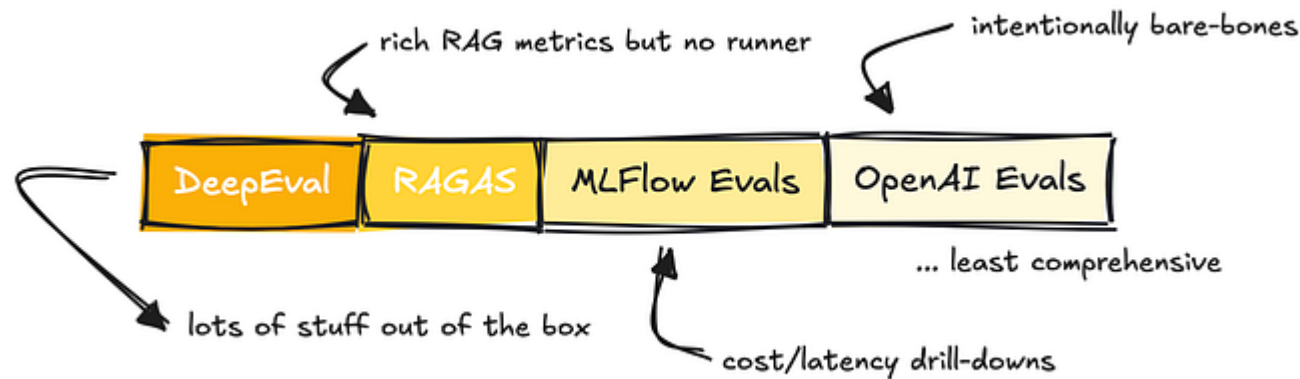
There are quite a few frameworks that help you out with evals, but I want to talk about a few popular ones: RAGAS, DeepEval, OpenAI's and MLFlow's Evals, and break down what they're good at and when to use what.



You can find the full list of different eval frameworks I've found in [this repository](#).

You can also use quite a few framework-specific eval systems, such as LlamaIndex, especially for quick prototyping.

OpenAI and MLFlow's Evals are add-ons rather than stand-alone frameworks, whereas RAGAS was primarily built as a metric library for evaluating RAG applications (although they offer other metrics as well).



However, it's important to mention that they all offer the ability to run evals on your own dataset, work for multi-turn, RAG, and agents in some way or another, support LLM-as-a-judge, allow setting up custom metrics, and are CI-friendly.

They differ, as mentioned, in how comprehensive they are.

MLFlow was primarily built to evaluate traditional ML pipelines, so the number of metrics they offer is lower for LLM-based apps. OpenAI is a very lightweight solution that expects you to set up your own metrics, although they provide an example library to help you get started.

DeepEval offers a lot out of the box, including the RAGAS metrics.

| Framework       | Pytest / CLI Runner                                                                                          | Metrics Ready-made                                                                             | Synthetic Data Gen                                                                                                      | Offline Judge                                                                                | Model-Agnostic                                                                       | Safety                                                                                  | Red-Team                                                                            | Custom Metrics (setup speed)                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| DeepEval        |  <code>deepeval test</code> | 40 +                                                                                           |  <code>deepeval create-dataset</code> |           |   |      |  |  G-Eval builder — minutes (one function)     |
| RAGAS           | ✗ (script asserts)                                                                                           | 6 core RAG +  |  KG-based Q-gen                       |           |   |  DIY | ✗                                                                                   |  AspectCritic one-liner — minutes            |
| MLflow Evaluate | ✗ <code>(mlflow.evaluate)</code>                                                                             | 3-4                                                                                            | ✗ BYO                                                                                                                   |  possible |   |      | ✗                                                                                   |  Subclass scorer — few lines, ~hour          |
| OpenAI Evals    |  CLI orchestrator          | ~10 templates                                                                                  |  helper script                        | ✗                                                                                            |  | ✗                                                                                       | ✗                                                                                   |  Full Python/YAML eval — flexible but slower |

Comparison between frameworks, see the Github repo [here](#) | Image by author

If we look at the metrics being offered, we can get a sense of how extensive these solutions are.

For example, faithfulness in one may mean the same as groundedness in another. Answer relevancy may be the same as response relevance, and so on.

This creates a lot of unnecessary confusion and complexity around evaluating systems in general.

Nevertheless, DeepEval stands out with over 40 metrics available and also offers a framework called G-Eval, which helps you set up custom metrics quickly making it the fastest way from idea to a runnable metric.

OpenAI's Evals framework is better suited when you want bespoke logic, not when you just need a quick judge.

According to the DeepEval team, custom metrics are what developers set up the most, so don't get stuck on who offers what metric. Your use case will be unique, and so will how you evaluate it.

**So, which should you use for what situation?**

MLFlow is a good choice if you're already invested in MLFlow or prefer built-in tracking and UI features. OpenAI's Evals framework is the most barebones, so it's best if you're tied into OpenAI infrastructure and want flexibility.

Lastly, DeepEval also provides red teaming via their DeepTeam framework, which automates adversarial testing of LLM systems. There are other frameworks out there that do this too, although perhaps not as extensively.

I'll have to do something on adversarial testing of LLM systems and prompt injections in the future. It's an interesting topic.

The dataset business is lucrative business which is why it's great that we're now at this point where we can use other LLMs to annotate data, or score tests.

However, LLM judges aren't magic and the evals you'll set up you'll probably find a bit flaky, just as with any other LLM application you build. According to the world wide web, most teams and companies sample-audit with humans every few weeks to stay real.

---

on your own.

It's good to know what the standard ones are though.

Hopefully it proved educational anyhow.

If you liked this one, be sure to read some of my other articles on [agentic frameworks](#) or building advanced [RAG agents](#).

You can follow me here, [LinkedIn](#) or my [website](#) if you want to get notified when I release something new.



[#data-science](#)

[#artificial-intelligence](#)

[#programming](#)

[#agentic-ai](#)

[#llm](#)